

Calcolabilità

Iniziamo introducendo il concetto di **Macchina di Turing**, queste corrispondono al modello astratto di computer, tutto ciò che è calcolabile per un computer lo è anche per una TM.

Una Turing Machine possiede un nastro infinito e una testina con la quale può leggere e scrivere sul nastro. Inizializziamo il nastro scrivendoci sopra la stringa di input, i restanti slot sono riempiti con il carattere *blank* $\sqcup$. I movimenti possibili per la testina sono *dx*, *sx*, *lettura*, *scrittura* e continua a computare fino al raggiungimento lo stato di accettazione o rifiuto, se non raggiunge nessuno dei due allora va in loop.

Esempio

Costruiamo una TM per il linguaggio

$$\{w\#w : w \in \{0, 1\}^*\}$$

La TM si muove a zig-zag tra i simboli a dx e sx di $\#$ e controlla se il simbolo a sx combacia con quello a dx, se si li cancella. Se trova un confronto non valido rifiuta altrimenti se sovrascrive tutti i simboli accetta.

Diamo adesso la definizione formale di TM.

Definizione - Turing Machine

Una TM è una tupla $M = (Q, \Sigma, \Gamma, \delta, q_{\text{start}}, q_{\text{acc}}, q_{\text{rej}})$ dove:

- Q è l'insieme finito degli stati
- Σ è l'insieme finito di input $\sqcup \notin \Sigma$
- Γ è l'alfabeto del nastro ($\sqcup \in \Gamma, \Sigma \subseteq \Gamma$)
- q_{acc} è lo stato di accettazione
- $q_{\text{rej}} \neq q_{\text{acc}}$ è lo stato di rifiuto
- $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ definita come, se $\delta(p, a) = (q, b, X)$ allora:
 - a viene letto dal nastro e sostituito da b
 - la macchina passa dallo stato p allo stato q
 - Il nastro viene spostato a sx se $X = L$ e a dx se $X = R$

La transizione avrà quindi come etichetta $a \rightarrow b; X$

Inoltre le transizione in realtà avvengono da $Q \setminus \{q_{\text{acc}}, q_{\text{rej}}\}$ dato che se la TM va in uno dei due termina subito.

Vediamo quindi adesso il concetto di **configurazione**. La configurazione di una TM si rappresenta in questo modo:

$$uqv$$

Dove:

- La posizione della testina è il primo simbolo di v
- q è lo stato corrente
- u è il restante contenuto del nastro precedente

Quindi ad esempio, una configurazione potrebbe essere:

$$1011q_701111$$

, inoltre la configurazione iniziale è $q_{\text{start}}w$ e quella finale $q \in \{q_{\text{acc}}, q_{\text{rej}}\}$.

Possiamo quindi dire che:

- $uq_i bv \vdash uq_j acv \Leftrightarrow \delta(q_i, b) = (q_j, c, L)$
- $uq_i bv \vdash uacq_j v \Leftrightarrow \delta(q_i, b) = (q_j, c, R)$

Diciamo che M **accetta** $w \in \Sigma^* \Leftrightarrow c_{\text{start}} = q_{\text{start}} w$ è t.c. $c_{\text{start}} \vdash^* c_{\text{acc}}$ dove c_{acc} è la configurazione di accettazione.

Definizione - Linguaggi Turing Riconoscibili

Un linguaggio L è Turing-Riconoscibile se $\exists$ TM M che lo riconosce, ovvero se $\forall w \in L, M$ accetta w .

Definizione - Decisore

Una TM è detta **decisore** se termina sempre, ovvero non va in loop.

Definizione - Decidibilità

Un linguaggio L è Turing-Decidibile se $\exists M$ decisore t.c. $L = L(M)$

Esistono delle varianti di TM che mantengono lo stesso potere espressivo:

- Nastro infinito a 2 direzioni
- TM che muovono a R, L, S - “stay-put”. Infatti basta che ad ogni transizione fanno prima uno spostamento a sinistra e poi a destra.
- TM con k nastri
- TM non deterministiche

TM Multinastro

Formalmente abbiamo che $\delta : Q \times \Gamma^k \rightarrow Q \times \Gamma^k \times \{L, R, S\}^k$ con $k = \#$ nastri, abbiamo quindi

$$\delta(q_i, a_1, \dots, a_k) = (q_j, b_1, b_k, L, R, \dots, L)$$

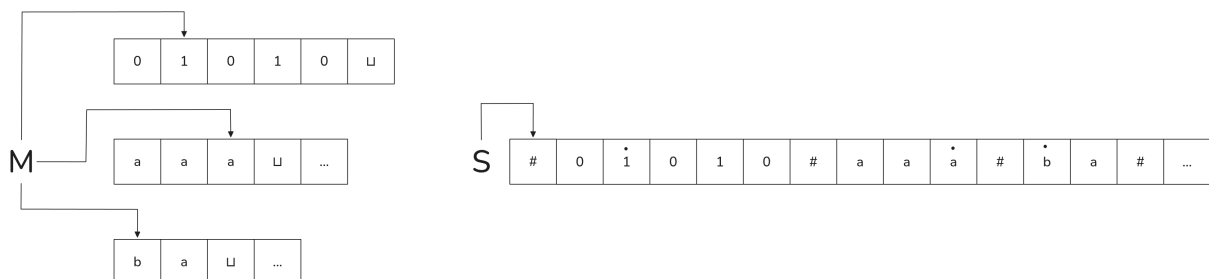
Se la macchina è nello stato q_i e le k testine leggono i simboli $a_1, \dots, a_k$ la macchina va nello stato q_j , scrive i simboli $b_1, \dots, b_k$ e ogni testina si muove come specificato.

Teorema

Ogni TM multinastro ha una TM singolo-nastro equivalente.

Dimostrazione - S simula M mettendo il suo contenuto in un singolo nastro in questo modo:

- Scrive sequenzialmente i contenuti dei k nastri con $\backslash \#$ come delimitatore
- Per tenere traccia di dove si trovano le testine, introduce un simbolo $\circ$ sopra al carattere corrispondente, le chiamiamo “testine virtuali”.



Quindi S su input $w = w_1, \dots, w_n$ si comporta così:

1. Pone il nastro nella configurazione "iniziale" $\#w_1^\circ w_2^\circ \dots w_n^\circ \# \sqcup^\circ \# \dots \#$
2. S scansiona il nastro dal primo all'ultimo $\#$, per capire quali siano i simboli puntati dalle testine virtuali. Poi fa una seconda scansione per aggiornare il nastro secondo δ' di M .
3. Se in qualsiasi momento una testina finisce su un $\#$ significa che ha raggiunto un $' \sqcup'$ e quindi vi scrive $' \sqcup'$ e sposta di una posizione a destra l'intero contenuto del nastro.

Corollario

Un linguaggio è Turing-Riconoscibile $\Leftrightarrow$ una TM multinastro lo riconosce.

Dimostrazione - Dato che la singolo-nastro è un caso speciale di multinastro e che ogni multinastro ha una singolo-nastro equivalente la dimostrazione è completa.

TM non Deterministica

Formalmente abbiamo che:

$$\delta : Q \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

Il concetto di non determinismo è uguale a quello che troviamo negli NFA. Semplicemente in questo caso quando la TM deve fare una mossa potrebbe avere più azioni possibili da svolgere e come per gli NFA, creiamo più percorsi indipendenti.

Simuliamo una NTM N con una TM D , per farlo consideriamo l'albero di computazione di N che esploriamo con una BFS. Non effettuiamo la ricerca con una DFS perché dato che vogliamo simulare il parallelismo se eseguiamo una DFS questa potrebbe continuare all'infinito su un ramo.

L'albero ha questa struttura:

- La radice è ε
- Ogni altro nodo ha un indirizzo $\in \Gamma_b = \{1, \dots, b\}$ con b il massimo numero di scelte tra tutti gli stati di N , nello specifico l'indirizzo è formato da xy con x indirizzo del padre e $y \in \Gamma_b$ "numero" del figlio, quindi ad esempio 231 indica il secondo figlio della radice $\rightarrow$ il suo terzo figlio $\rightarrow$ il suo primo figlio.

La TM D che simula N è una multi-nastro con 3 nastri:

1. Il nastro 1 contiene l'input w e non viene mai alterato. Ci serve come riferimento per quando cambiamo ramificazione.
2. Il nastro 2 simula un ramo della computazione di N , quando ha più scelte effettua quella presente in 3.
3. Il nastro 3 contiene l'indirizzo del nodo fino a cui simulare, quindi conterrà ad esempio "1", "2", "3" poi passa ai figli del primo nodo quindi "11", "12", "13" e poi a quelli del primo nodo ancora "111", "112"...

La TM D si comporta quindi in questo modo:

1. Inizialmente il nastro 1 contiene w e gli altri 2 sono vuoti.
2. Copia il nastro 1 sul nastro 2
3. Usa il nastro 2 per simulare N con input w . Ad ogni step N consulta il prossimo simbolo sul nastro 3
 - Se il nastro 3 è vuoto va allo step 4
 - Se la simulazione rifiuta va allo step 4
 - Se la simulazione accetta, accetta
4. Sostituisce la stringa sul nastro 3 con la successiva in ordine lessicografico. Va al passo 2.

Esempio

Una TM non deterministica può essere una TM che su input binario:

- input 0:
 - Si muove a dx
 - Si muove a sx
- input 1: accetta

Quindi quando legge uno 0 crea due rami, se simuliamo questa TM non deterministica con una deterministica avremo che eseguirà entrambe le scelte prima o poi.

Tesi di Church-Turing

Qualunque problema calcolabile da un algoritmo può essere risolto da una macchina di Turing.

Linguaggi Decidibili

Sono quei linguaggi per i quali esiste una TM che per ogni stringa termina sempre in un tempo finito e stabilisce se questa appartiene o no al linguaggio.

Problema dell'accettazione per i DFA - Definiamo

$$A_{\text{DFA}} = \{ \langle B, w \rangle \mid B \text{ è un DFA che accetta } w \}$$

E presentiamo una TM che accetta A_{DFA} :

M = "Su input $\langle B, w \rangle$, con B DFA e w stringa:

- Simula B su input w
- Se la simulazione finisce con uno stato di accettazione, accetta; altrimenti rifiuta."

Problema dell'accettazione per gli NFA - Definiamo

$$A_{\text{NFA}} = \{ \langle B, w \rangle \mid B \text{ è un NFA che accetta } w \}$$

E presentiamo una TM che accetta A_{NFA} :

N = "Su input $\langle B, w \rangle$, con B NFA e w stringa:

- Converti B in un DFA C
- Esegui la TM 'M' (per A_{DFA}) su C

-Se M accetta, accetta; altrimenti rifiuta."

Determinare se una regex genera una stringa - Definiamo

$$A_{\text{REX}} = \{ \langle R, w \rangle \mid R \text{ regex che genera } w \}$$

E costruiamo una TM che accetta A_{REX} :

P = "Su input $\langle R, w \rangle$ con R regex e w stringa:

- Converti R in NFA A
- Esegui la TM N (per A_{NFA} su A)
- Se N accetta, accetta; altrimenti rifiuta"

Emptiness-testing per i DFA - Definiamo

$$E_{\text{DFA}} = \{ \langle A \rangle \mid A \text{ DFA e } L(A) = \emptyset \}$$

E costruiamo una TM che accetta E_{DFA} :

T = "Su input $\langle A \rangle$ con A DFA:

- Marca lo stato iniziale
- Finchè non vengono marcati nuovi stati, marca ogni stato che ha una transizione entrante da uno stato già marcato.
- Se almeno uno stato accettante è stato marcato, rifiuta; altrimenti accetta."

Determinare se due DFA accettano lo stesso linguaggio - Definiamo

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A \text{ e } B \text{ DFA e } L(A) = L(B) \}$$

Utilizziamo la differenza simmetrica:

$$L(C) = L(A) \triangle L(B)$$

, se è vuota abbiamo che $L(A) = L(B)$. Si ha

$$L(C) = (L(A) \cap \overline{L(B)}) \cup (\overline{L(A)} \cap L(B))$$

Dobbiamo verificare se $\langle C \rangle \in E_{DFA}$, costruiamo

F = "Dato un input:

- Costruisci il DFA C come descritto
- Simula la TM T (per E_{DFA})
- Se T accetta, accetta; altrimenti rifiuta"

Determinare se una CFG genera una stringa - Definiamo

$$A_{CFG} = \{ \langle G, w \rangle \mid G \text{ CFG genera } w \}$$

Costruiamo la TM

M = "Su input $\langle G, w \rangle$ con G CFG e w stringa:

- Converti G in CNF (Chomsky)
- Enumera le derivazioni con $2n - 1$ passi dove $n = |w|$, se $n = 0$ enumera solo quelle con 1 passo. Da questa possiamo ricavare il successivo *claim*
- Se una delle derivazioni genera w , accetta; altrimenti rifiuta."

Claim - Sia $G = (V, \Sigma, R, S)$ una CFG in CNF. Data $w \in G$ con $|w| = n$, la sua derivazione è di $2n - 1$ passi.

Dimostrazione - Dimostriamolo per induzione su n .

- Caso Base: $n = 1$ allora $|w| = 1$ significa che $w \in \Sigma$. Quindi w è derivata con la regola $S \rightarrow w$ in $1 = 2 \cdot 1 - 1$ passi.
- Passo Induttivo: Assumiamo sia vero per stringhe di lunghezza $\leq n - 1$. Sia $w \in G$ t.c. $|w| = n$. Visto che G è in CNF, $S \xRightarrow{*} w$ implica che $S \rightarrow AB \xRightarrow{*} w$. Quindi possiamo scrivere $w = xy$ con $A \xRightarrow{*} x$ e $B \xRightarrow{*} y$. Visto che non ci sono ε -regole in CNF abbiamo che $x, y \neq \varepsilon$. Quindi siano $|x| = k$ e $|y| = n - k$, per ipotesi induttiva x, y sono derivabili in $2k - 1$ e $2(n - k) - 1 = 2n - 2k - 1$ produzioni. Quindi $AB \xRightarrow{*} w$ ha $2k - 1 + 2n - 2k - 1 = 2n - 2$ produzioni, aggiungendo $S \rightarrow AB$ otteniamo che $S \xRightarrow{*} w$ in $2n - 1$ produzioni.

Emptiness-testing per CFG - Definiamo

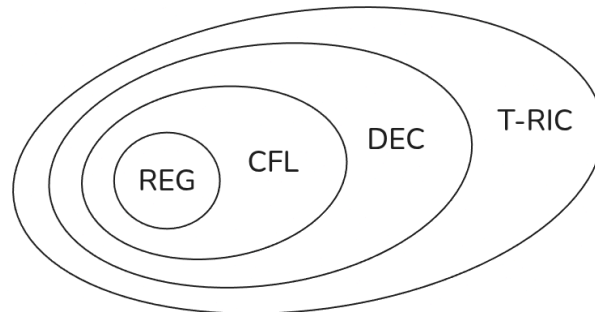
$$E_{CFG} = \{ \langle G \rangle \mid G \text{ CFG e } L(G) = \emptyset \}$$

Costruiamo la TM

M = "Su input G, dove G è una CFG:

- Marca tutti i terminali in Σ
- Finchè non vengono marcate nuove variabili:
 - Marca ogni $A \in V$ per cui $\exists A \rightarrow U_1 U_2 \dots U_l \in R$ dove ogni U_i è già stato marcato
- Se S è marcata, rifiuta; altrimenti accetta“.

Otteniamo quindi questa situazione per i linguaggi:



Esistono però linguaggi non TURING-DEC / RIC, introduciamo quindi il problema A_{TM} che determina se una TM accetta un determinato input.

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ TM e } w \in L(M) \}$$

Definizione - Turing Machine Universale

Una TM universale è una TM in grado di simulare ogni altra TM. Si dice infatti che un modello di calcolo è Turing-complete se è equivalente ad una TM universale.

Una qualsiasi TM M può essere completamente descritta da un insieme finito di transizioni, e quindi può essere codificata come una stringa finita di simboli. Una TM universale prende in input la descrizione codificata della TM da simulare $\langle M \rangle$ e l'input w su cui dovrebbe lavorare e simula M .

Notiamo che A_{TM} è *T-REC*, infatti abbiamo la TM U che lo riconosce:

U = “Su input $\langle M, w \rangle$ con M TM e w stringa:

- Simula M su w
- Se M raggiunge uno stato accettante, accetta; altrimenti rifiuta“

Con questa costruzione se M va in loop ci andrà anche U .

Teorema

A_{TM} non è decidibile.

Ma per dimostrarlo abbiamo prima bisogno di un altro teorema.

Teorema

Esistono linguaggi non Turing-riconoscibili.

Dimostrazione - Notiamo che l'insieme di tutte le TM $\mathcal{M}$ non è numerabile in quanto l'insieme di tutte le stringhe Σ^* su un alfabeto Σ è numerabile, queste rappresentano gli encoding di ogni TM in stringe $\langle M \rangle$.

Notiamo anche che l'insieme delle stringhe binarie infinite $\mathcal{B}$ non è numerabile. Adesso consideriamo $\mathcal{L} = \mathcal{P}(\Sigma^*)$ l'insieme di tutti i linguaggi definiti su Σ . Mostriamolo fornendo una corrispondenza biunivoca con $\mathcal{B}$, quindi ogni linguaggio $A \in \mathcal{L}$ corrisponde ad una sequenza univoca in $\mathcal{B}$.

Definiamo il concetto **Sequenza Caratteristica** di A , ovvero una sequenza binaria definita come segue:

$$\mathcal{X}_A = b_1 b_2 \dots \text{ t.c. } b_i = \begin{cases} 1 & s_i \in A \\ 0 & s_i \notin A \end{cases}$$

Esempio

Prendiamo il linguaggio A = "Stringhe binarie che iniziano con 0":

- $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, \dots\}$
- $A = \{\epsilon, 0, 00, 01, \dots\}$
- $\mathcal{X}_A = 0101100\dots$

Notiamo che $f : \mathcal{L} \rightarrow \mathcal{B} : A \rightarrow \mathcal{X}_A$ è biunivoca, perciò visto che $\mathcal{B}$ non è numerabile non lo è neanche $\mathcal{L}$. Ovvero $\exists A \in \mathcal{L}$ t.c. $\nexists M \text{ TM t.c. } L(M) = A$.

Quindi la funzione $h : \mathcal{M} \rightarrow \mathcal{L} : M \rightarrow L(M)$ non è biiettiva, quindi $\exists L \in \mathcal{L}$ t.c. $\nexists M$ per cui $L(M) = L$.

Adesso possiamo tornare a dimostrare che A_{TM} non è decidibile.

Dimostrazione - Supponiamo per assurdo che $A_{\text{TM}} \in \text{DEC}$ e sia H un decisore per A_{TM} :

$$H(\langle M, w \rangle) = \begin{cases} \text{accept} & \text{se } M \text{ accetta } w \\ \text{reject} & \text{se } M \text{ non accetta } w \end{cases}$$

Adesso costruiamo una TM

D = "Su input $\langle M \rangle$ con M TM:

1. Esegui H con input $\langle M, \langle M \rangle \rangle$
2. Dai in output l'opposto dell'output di H

$$D(\langle M \rangle) = \begin{cases} \text{accetta} & \text{se } M \text{ non accetta } \langle M \rangle \\ \text{rifiuta} & \text{se } M \text{ accetta } \langle M \rangle \end{cases}$$

Ma adesso cosa succede se eseguiamo D con la sua stessa descrizione $\langle D \rangle$ in input?

$$D(\langle D \rangle) = \begin{cases} \text{accetta} & \text{se } D \text{ non accetta } \langle D \rangle \\ \text{rifiuta} & \text{se } D \text{ accetta } \langle D \rangle \end{cases}$$

Ma abbiamo ottenuto una contraddizione, se D accetta $\langle D \rangle$ allora D deve rifiutare $\langle D \rangle$ e viceversa.

Definizione - Co-Turing-Riconoscibile

Un linguaggio si dice *Co-Turing-Riconoscibile* se è il complemento di un linguaggio *Turing-Riconoscibile*.

Teorema

Un linguaggio L è decidibile $\Leftrightarrow L \in \text{REC}$ e $L \in \text{CO-REC}$

Dimostrazione

- $\Rightarrow$ se L è decidibile, $\bar{L}$ è decidibile quindi L e $\bar{L}$ sono *Turing-Riconoscibili*.
- $\Leftarrow$ sia M_1 un riconoscitore per A e M_2 un riconoscitore per $\bar{A}$

Costruiamo $M =$ “Su input w :

1. Esegui M_1 e M_2 in parallelo su w
2. Se M_1 accetta, accetta; se M_2 accetta, rifiuta“

Ogni stringa w è in A oppure in $\bar{A}$, quindi uno tra M_1 e M_2 deve accettare. Visto che M si ferma se M_1 o M_2 accettano, termina sempre e quindi è un decisore. Inoltre, accetta le stringhe in A e rifiuta quelle in $\bar{A}$ quindi è un decisore per A .

Corollario 1

Se L non è decidibile allora almeno uno tra L e $\bar{L}$ non è *Turing-Riconoscibile*.

Corollario 2

$\bar{A}_{\text{TM}}$ non è *Turing-Riconoscibile*.

Riducibilità

Vediamo una tecnica per dimostrare che alcuni problemi non sono decidibili o riconoscibili. Una **riduzione** è un modo di convertire un problema in un altro problema in modo che una soluzione per il secondo problema possa essere usata per il primo. *(Ci serve aumentare il modello delle TM aggiungendo un nastro di output WLOG)*

Diremo che una TM calcola una funzione su un dato input se inizia con w sul nastro di input e termina con $f(w)$ sul nastro di output.

Definizione - Calcolabilità

Una funzione $f : \Sigma^* \rightarrow \Sigma^*$ è calcolabile se $\exists \text{TM } M$ che calcola $f \forall w \in \Sigma^*$